# Incident Investigation Report: Host DESKTOP-5OG7C28 (Wazuh agent 001)

**Evidence examined:** Scenario_1_Alerts.json (5 alerts) and Scenario_1_Archives.json (32 events: 24 Sysmon records 27932–27955 and 8 Security records 83312–83319). **Time convention:** All times are UTC, taken from Sysmon UtcTime or Security SystemTime. Wazuh `timestamp` values are local time (UTC+2) and run about 0.9 s behind the source events because of ingestion delay.

## 1. Executive Summary

Between 13:37:25 and 13:37:27 UTC, an interactive PowerShell console (PID 1392) belonging to the local user **Darragh Casey** ran a rapid automated sequence:

1. It executed `hostname.exe` and `whoami.exe`.
2. It started a child PowerShell (PID 10880) that tested whether `C:\AtomicRedTeam\ExternalPayloads\Scenario_1_script.bat` existed.
3. It started a second child PowerShell (PID 4500) that launched the batch file with `Start-Process`.
4. That launch produced `cmd.exe` (PID 6000), which executed the script.

This chain is the only activity in the dataset of investigative significance. The directory naming is consistent with an adversary-emulation test framework, which matches the isolated lab context.

The investigation cannot determine what the batch script did. Sysmon recorded no child processes, file creations or other events attributable to `cmd.exe` PID 6000. The Sysmon record numbers are contiguous, so no Sysmon events are missing from the supplied window. The data shows no privilege escalation, persistence, credential access, lateral movement or network activity, though the collection limits in Section 2 restrict how much that absence means.

Of the five Wazuh alerts, only rule **92004** (level 4, "PowerShell spawned Windows command shell") correctly characterises the activity of interest. The other four alerts carry unsupported technique labels:

- The two **level-15** alerts (rule 92213, "Ingress Tool Transfer") were triggered by temporary policy-test files that PowerShell creates itself.
- The two **level-6** alerts (rule 92217, "Lateral Tool Transfer") were triggered by a routine Microsoft Defender signature and platform update.

In this dataset, alert severity ran inversely to investigative relevance.

## 2. Scope of Activity

**Host:** DESKTOP-5OG7C28, a WORKGROUP (non-domain) system, Wazuh agent 001 "Windows_VM", IP 192.168.232.130, Windows build 10.0.26100. Manager: dcasey-virtual-machine.

**Window:** 13:33:25.413 to 13:41:45.610 UTC (about 8 min 20 s). The activity of interest occupies about 2 seconds (13:37:25.182 to 13:37:27.231).

**Accounts:**

- *DESKTOP-5OG7C28\Darragh Casey*: Logon ID 0xFCD57, terminal session 1, all processes at Medium integrity.
- *NT AUTHORITY\SYSTEM*: Logon ID 0x3E7, used by Windows services.

**Event mix:** 20 Sysmon EID 1 (process create), 4 Sysmon EID 11 (file create), 4 Security 4624 (logon), 4 Security 4624's companion 4672 (special privileges).

**Collection limitations that affect interpretation:**

- *No network logging visible.* There are no Sysmon EID 3 events, yet Defender ran with `-HttpDownload` and wrote downloaded executables, so network connections must have occurred. Sysmon network logging is therefore disabled or filtered, and the absence of network events says nothing about the batch script.
- *No process-termination logging.* There are no EID 5 events, even for the short-lived `hostname`/`whoami` processes. Process termination is not collected, so the script's runtime and exit cannot be established.
- *Other event types absent.* There are no EID 7, 13 or 22 events (image loads, registry, DNS) and no PowerShell script block logging (4104).
- *Filtered file logging.* EID 11 records carry RuleName "EXE" or "-", indicating filtered file-creation logging. File creations by the script could have gone unrecorded.
- *No batch file record.* Neither the batch file's creation nor its hash appears in the window, so its contents and provenance are unknown.
- *Earlier activity outside the data.* The `firedtimes` counters show these rules fired before the supplied window: 92217 at 6–7, 92213 at 5–6, 92004 at 2. Similar activity occurred earlier on this manager but was not supplied.

## 3. Detailed Investigation

### 3.1 Activity of interest: scripted batch-file execution

**Observed process lineage:**

```
powershell.exe PID 1392  (no arguments; interactive console; CWD of children: C:\Users\Darragh Casey\Desktop\s_1\)
 ├─ HOSTNAME.EXE PID 1572   13:37:25.182
 ├─ HOSTNAME.EXE PID 8868   13:37:25.284
 ├─ HOSTNAME.EXE PID 5276   13:37:25.352
 ├─ whoami.exe   PID 5856   13:37:25.386
 ├─ powershell.exe PID 10880 13:37:25.589  & {if (Test-Path "...\Scenario_1_script.bat") {exit 0} else {exit 1}}
 │    └─ [EID 11] __PSScriptPolicyTest_qhypv0d4.v5c.ps1   13:37:25.773  (alert 92213)
 ├─ HOSTNAME.EXE PID 7596   13:37:26.099
 ├─ whoami.exe   PID 3892   13:37:26.124
 └─ powershell.exe PID 4500 13:37:26.282  & {Start-Process "...\Scenario_1_script.bat"}
      ├─ [EID 11] __PSScriptPolicyTest_5aj3nptp.trx.ps1   13:37:26.445  (alert 92213)
      └─ cmd.exe PID 6000  13:37:27.087  /c ""C:\AtomicRedTeam\ExternalPayloads\Scenario_1_script.bat" "  (alert 92004)
           └─ no child process or file events recorded
OpenConsole.exe PID 9060   13:37:27.231  -Embedding, parent PID 944 (unresolved)
```

**Observed facts about the chain:**

- *Single session.* Every process in the chain shares Logon ID 0xFCD57, LogonGuid {4b2ce92d-5450-6adb-57cd-0f0000000000}, terminal session 1 and Medium integrity.
- *Path normalisation.* The script path given to PowerShell was `C:\AtomicRedTeam\atomics\..\ExternalPayloads\Scenario_1_script.bat`. The `cmd.exe` command line shows the normalised form `C:\AtomicRedTeam\ExternalPayloads\Scenario_1_script.bat`, so both refer to the same file.
- *Working directories.* Both child PowerShell processes ran with working directory `C:\Users\DARRAG~1\AppData\Local\Temp\`, unlike the parent console, whose children inherited `Desktop\s_1`.

**Inferences:**

- *Check-then-execute pattern.* A file-existence check followed 0.7 s later by execution of the same file is characteristic of a test harness performing a prerequisite check before running a payload. *High confidence.*
- *The file existed.* `Start-Process` produced a `cmd.exe` process for the file. The `cmd.exe /c ""<file>" "` form is how Windows launches a batch file through its file association. *High confidence.*
- *Automated, not manual.* Three `hostname.exe` executions within 170 ms, followed by `whoami.exe` 34 ms later, are not consistent with human typing. *High confidence.*
- *Harness context-logging, not batch payload.* A hostname/whoami pair immediately precedes each child PowerShell launch, roughly 200 ms and 160 ms before. This suggests the automation records host and user context before each step. Deliberate discovery as part of the scenario cannot be excluded because command output is not captured. Either way, these commands came from the parent console, not from the batch file. *Medium confidence.*
- *Console host.* `OpenConsole.exe` (Windows Terminal console host, `-Embedding`, DCOM-style launch) started 144 ms after `cmd.exe` 6000. It is assessed as the console window created to host the batch file launched by `Start-Process`. *Medium-high confidence; based on timing only, with no direct parent link.*
- *Folder naming.* The `Desktop\s_1` working-directory name parallels "Scenario_1". This is circumstantial.

### 3.2 Session context derived from Sysmon GUIDs

Sysmon ProcessGuid and LogonGuid values embed a creation timestamp (bytes 5–8, read as Data3+Data2). This was validated against the dataset itself:

- PID 1572's GUID {4b2ce92d-**6315-6adb**-…} gives 0x6ADB6315 = 1792762645 = 13:37:25 UTC, matching its UtcTime.
- `cmd.exe` 6000 ({…-6317-6adb-…}) gives 13:37:27, also matching.

Applying the same decoding to other GUIDs gives an approximate session history:

| GUID source | Decoded time (UTC) | Meaning |
|---|---|---|
| services.exe / SYSTEM logon {…-5414-6adb-…} | ~12:33:24 | System start |
| MsMpEng.exe {…-541d-6adb-…} | ~12:33:33 | Defender engine start |
| User LogonGuid {…-5450-6adb-…} | ~12:34:24 | User logon |
| PowerShell 1392 {…-5510-6adb-…} | ~12:37:36 | Console opened |

The console had therefore been open for about 60 minutes before the test ran. This is inferred and depends on the GUID encoding, but it is internally consistent.

### 3.3 Microsoft Defender update and alert 92217

**Observed:**

- `MsMpEng.exe` (PID 3132, SYSTEM) spawned three `MpCmdRun.exe` instances from `C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26090.9-0\`. All three share SHA256 C0C9670D…EA12315:
  - PID 9204, `SignatureUpdate -ScheduleJob -RestrictPrivileges`
  - PID 5128, `SignaturesUpdateService -ScheduleJob -UnmanagedUpdate`
  - PID 4396, `SignaturesUpdateService -ScheduleJob -HttpDownload -RestrictPrivileges`
- PID 4396 created `mpam-bce1d755.exe` and `UpdatePlatform.exe` in `C:\Windows\SystemTemp\<GUID>MpCommU\` folders.

- Rule 92217 fired on both files, labelling them "Executable dropped in Windows root folder" with mapping T1570, Lateral Tool Transfer.

**Assessment: benign; the technique mapping is unsupported.**

- *Wrong location.* The files are in a subfolder of `C:\Windows`, not its root.
- *Wrong origin.* The writer is Defender's own updater, running under its legitimate parent, with update arguments that explicitly request an HTTP download.
- *No lateral movement.* There is no remote logon (all 4624 events are Type 5 service logons), no network share path and no remote source.
- *Unrelated to the test.* This activity ended about 3 min 45 s before the activity of interest. It used a different account and a different process tree, with no linkage.

**Caveat:** EID 11 does not record hashes or signatures for the dropped files. Definitive confirmation would require hashing or Authenticode-checking them if they are still present.

The two `SecurityHealthHost.exe` launches at 13:33:25 (user context, `-Embedding`, unresolved parent PID 944) immediately precede the Defender update. They are consistent with Windows Security component activity and are considered benign.

### 3.4 PowerShell policy-test files and alert 92213

**Observed:** Each child PowerShell (PIDs 10880 and 4500) created a randomly named `__PSScriptPolicyTest_*.ps1` file in the user's Temp folder, 184 ms and 163 ms after starting. Rule 92213 (level 15, emailed) labelled these as T1105, Ingress Tool Transfer.

**Assessment:** PowerShell itself creates these files at start-up to test script-execution policy. Their naming, timing and creator are consistent with that behaviour. There is no evidence of any external transfer, so the T1105 mapping is unsupported.

These alerts are nonetheless causally tied to the activity of interest: they fired because the harness spawned PowerShell. They are best treated as indirect indicators of PowerShell launches, not as evidence of tool transfer.

### 3.5 Background activity excluded as unrelated

The following events are all SYSTEM-context Windows maintenance from standard Microsoft paths with expected arguments, and none links to the user's process tree:

- `svchost.exe -k GPSvcGroup` at 13:35:14 (Group Policy service, parent `services.exe`)
- `MicrosoftEdgeUpdate.exe /ua /installsource scheduler` at 13:37:53
- `UsoClient.exe StartWork` at 13:39:43
- `MoUsoCoreWorker.exe` at 13:39:43 and 13:41:44 (parent: svchost UsoSvc)

The Edge Update and UsoClient parents (PID 8) are unresolved by Sysmon. This is unusual but not suspicious in itself, and the command lines are consistent with scheduled tasks.

All four 4624/4672 pairs are Type 5 SYSTEM logons through `services.exe`, using Advapi/Negotiate with no network fields. Each coincides with a service start (13:33:26.6, 13:35:14.4, 13:39:44.6, 13:41:45.6).

The `cmd.exe` at 13:37:27 is not unrelated: it runs at the same time as this background activity but belongs to the user's process tree.

### 3.6 What was not observed

The data contains no evidence of:

- any process spawned by `cmd.exe` 6000
- files written by the script
- elevation (no High or System integrity in the user chain)
- new logons of Type 2, 3 or 10
- persistence mechanisms
- credential access
- connections to other hosts

Two caveats limit what this absence means:

- *Built-ins are invisible.* Batch built-ins (echo, set, copy, del, type, directory changes) execute inside `cmd.exe` without creating processes, so they would not appear in Sysmon EID 1.
- *Other event types are unavailable.* Network, registry and termination events are not available (Section 2).

## 4. Evidence Analysis

| Finding | Supporting records and values |
|---|---|
| Interactive console is the origin | Records 27940–27948: ParentProcessId 1392, ParentProcessGuid {4b2ce92d-5510-6adb-fd01-…}, ParentCommandLine `"…\powershell.exe"` (no arguments) |
| Existence check of | Record 27944: PID 10880, `& {if (Test-Path \"C:\AtomicRedTeam\atomics\..\ExternalPayloads\Scenario_1_script.bat\") {exi` |

| | |
|---|---|
| payload | `t 0} else {exit 1}}` |
| Payload launched | Record 27948: PID 4500, `& {Start-Process \"…\Scenario_1_script.bat\"}` |
| Batch executed by cmd | Record 27950 / alert 1792762647.1728391: `cmd.exe` PID 6000, ParentProcessId 4500, CommandLine `/c ""C:\AtomicRedTeam\ExternalPayloads\Scenario_1_script.bat" "`, SHA256 15CD209C…E0D7B, IntegrityLevel Medium |
| Automated context commands | Records 27940–27943, 27946–27947: HOSTNAME.EXE SHA256 F8B20772…C747C9; whoami.exe SHA256 23240EF9…F56CD3; sub-second spacing |
| Console host for script window | Record 27951: OpenConsole.exe PID 9060, `-Embedding`, 144 ms after record 27950 |
| PowerShell policy-test artifacts | Records 27945, 27949 (alerts 92213): TargetFilename `…\Temp\__PSScriptPolicyTest_qhypv0d4.v5c.ps1` and `…_5aj3nptp.trx.ps1`, Image `powershell.exe` PIDs 10880/4500 |
| Defender update | Records 27934, 27935, 27936: parent MsMpEng.exe PID 3132, SYSTEM, update arguments; records 27937–27938 (alerts 92217): files under `C:\Windows\SystemTemp\…MpCommU\` |
| No missing Sysmon data | Sysmon EventRecordIDs 27932–27955 and Security 83312–83319 are both unbroken sequences |
| No remote or interactive logons in window | Records 83312, 83314, 83316, 83318: LogonType 5, ProcessName services.exe, no source address |
| No observed script follow-on | No record after 27950 has ParentProcessId 6000 or Image `cmd.exe` |

## 5. Timeline of Events

| Time (UTC) | Record | Event | Assessment |
|---|---|---|---|
| ~12:33:24 | (GUID-derived) | System and SYSTEM logon session start | Inferred context |
| ~12:34:24 | (GUID-derived) | User Darragh Casey logon session 0xFCD57 | Inferred context |
| ~12:37:36 | (GUID-derived) | PowerShell console PID 1392 started | Inferred context |
| 13:33:25.413 / .628 | 27932, 27933 | SecurityHealthHost.exe PIDs 5424, 6508 (user, -Embedding) | Benign |
| 13:33:26.333 | 27934 | MpCmdRun PID 9204 SignatureUpdate | Benign, Defender |
| 13:33:26.495 | 27935 | MpCmdRun PID 5128 SignaturesUpdateService -UnmanagedUpdate | Benign, Defender |
| 13:33:26.617 | 83312, 83313 | 4624 Type 5 SYSTEM + 4672 | Benign, service logon |
| 13:33:39.753 | 27936 | MpCmdRun PID 4396 -HttpDownload | Benign, Defender |
| 13:33:39.880 | 27937 | mpam-bce1d755.exe created; alert 92217 | False-positive mapping |
| 13:33:39.943 | 27938 | UpdatePlatform.exe created; alert 92217 | False-positive mapping |
| 13:35:14.448 / .450 | 83314, 83315, 27939 | Type 5 logon; svchost GPSvcGroup | Benign |
| 13:37:25.182–.352 | 27940–27942 | hostname.exe ×3 from PS 1392 | Automated; relevant |
| 13:37:25.386 | 27943 | whoami.exe from PS 1392 | Automated; relevant |
| 13:37:25.589 | 27944 | PowerShell 10880 Test-Path on batch file | **Relevant: prerequisite check** |
| 13:37:25.773 | 27945 | PSScriptPolicyTest file; alert 92213 | Benign PowerShell artifact |
| 13:37:26.099 / .124 | 27946, 27947 | hostname.exe, whoami.exe | Automated; relevant |
| 13:37:26.282 | 27948 | PowerShell 4500 Start-Process batch file | **Relevant: execution** |
| 13:37:26.445 | 27949 | PSScriptPolicyTest file; alert 92213 | Benign PowerShell artifact |
| 13:37:27.087 | 27950 | cmd.exe 6000 runs Scenario_1_script.bat; alert 92004 | **Relevant: payload execution** |
| 13:37:27.231 | 27951 | OpenConsole.exe 9060 | Likely console for cmd 6000 |
| 13:37:53.569 | 27952 | MicrosoftEdgeUpdate.exe (scheduler) | Benign |
| 13:39:43.850 / .908 | 27953, 27954 | UsoClient StartWork; MoUsoCoreWorker | Benign |
| 13:39:44.638 | 83316, 83317 | Type 5 logon + 4672 | Benign |
| 13:41:44.837 | 27955 | MoUsoCoreWorker | Benign |
| 13:41:45.609 | 83318, 83319 | Type 5 logon + 4672 | Benign |

## 6. MITRE ATT&CK Mapping

| Tactic | Technique | Evidence | Basis |
|---|---|---|---|
| Execution | T1059.001 Command and Scripting Interpreter: PowerShell | PIDs 10880 and 4500 execute inline script blocks (`& {…}`) spawned from PS 1392 (records 27944, 27948) | Observed |
| Execution | T1059.003 Command and Scripting Interpreter: Windows Command Shell | `cmd.exe /c` executing a .bat file, parent PowerShell 4500 (record 27950, alert 92004) | Observed |
| Discovery | T1082 System Information Discovery | `hostname.exe` ×4 (records 27940–27942, 27946) | Execution observed; discovery intent uncertain (may be harness logging) |
| Discovery | T1033 System Owner/User Discovery | `whoami.exe` ×2 (records 27943, 27947) | Same as above |

**Mappings assessed as not supported:**

- *T1105 Ingress Tool Transfer (rule 92213):* the files are PowerShell's own policy-test artifacts, with no evidence of external transfer.

- *T1570 Lateral Tool Transfer (rule 92217):* the files were written locally by Defender's updater, with no remote session or share.
- *T1083 File and Directory Discovery:* considered for the Test-Path check and rejected, because the check verifies the harness's own payload rather than exploring the environment.
- *T1204 User Execution:* rejected, because the launch was programmatic.

**Unknown:** any technique implemented inside Scenario_1_script.bat cannot be mapped, because its contents and effects are not in the telemetry.

## 7. Overall Investigation Conclusion

**Observed facts:**

- Within one interactive session of local user Darragh Casey on DESKTOP-5OG7C28, an automated sequence ran from a long-open PowerShell console.
- The sequence ran `hostname.exe` and `whoami.exe`, verified that `C:\AtomicRedTeam\ExternalPayloads\Scenario_1_script.bat` existed, and executed it through PowerShell `Start-Process`, resulting in `cmd.exe` PID 6000 at 13:37:27.087 UTC.
- All activity ran at Medium integrity.
- No events attributable to the script's execution were recorded, and the Sysmon record sequence is complete for the window.
- Four of five alerts were generated by benign mechanisms: Defender updating itself, and PowerShell creating policy-test files.

**Inferences:**

- The sequence is a scripted adversary-emulation or test-harness execution of a scenario batch file, consistent with the isolated lab context. The check-then-execute structure and sub-second automation indicate a framework, not hands-on-keyboard activity.
- The hostname/whoami calls are most likely harness context-logging rather than part of the payload.
- `OpenConsole.exe` hosted the script's console window.
- The Defender and Windows Update activity is unrelated background noise.

**Significance:** The investigation confirms that the batch file was executed, but cannot assess its impact. Whatever the script did either used `cmd.exe` built-ins or produced activity outside the Sysmon configuration's coverage.

**Detection-engineering findings from this dataset:**

- The highest-severity alerts (level 15, emailed) were false positives in their technique labelling, while the one accurate alert was level 4.
- Rule 92213 would benefit from excluding `__PSScriptPolicyTest_*.ps1` created by `powershell.exe`.
- Rule 92217 would benefit from excluding `MpCmdRun.exe` writes to `C:\Windows\SystemTemp\*MpCommU\`.
- Enabling Sysmon EID 3/5, PowerShell script block logging (4104), and command-line capture of batch contents (or hashing/archiving payload files) would allow the script's behaviour to be assessed in future runs.

**Confidence:**

- **High** for the reconstructed execution chain and for the benign dispositions of the Defender and policy-test alerts.
- **Medium** for the interpretation of hostname/whoami as harness activity and for the OpenConsole association.
- **Low** for any statement about the batch script's actual effects, which the telemetry does not show.

**Overall confidence: Medium.** The sequence of events is well established, but the central question of what the executed script did cannot be answered from the supplied evidence.